

Real-Time Fraud Detection & Prevention in Digital Lending Ecosystems

AI-Powered Detection · Behavioral Analytics · Explainable Decisioning

A transaction-type-aware pipeline that classifies each event, activates only the relevant fraud signals, blends deterministic ML with vector similarity, and returns an explainable real-time decision.

TEAM

Abhinav Pal

2022B2A11556G · BITS Pilani, Pilani Campus

HACKATHON

Synchrony Hackathon
Problem Statement PS-2

DATE

September 2026

Source code: github.com/abhinav-pal246/Synchrony_Real_Fraud_Detection_and_Prevention_In_digital_Landing

THE PROBLEM

Static rules cannot keep pace with evolving fraud

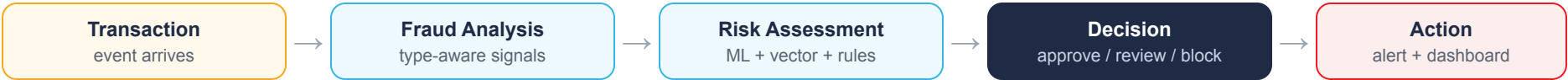
Why traditional detection breaks down

- Digital lending drives **high-volume, low-latency** transaction streams.
- Fraud patterns evolve continuously — rules lag behind **novel** attacks.
- Applying **one model / one feature-set to every transaction** is inefficient and context-blind.
- Aggressive rules raise **false positives** that block genuine customers.
- Analysts get a "fraud" flag with **no reason** — no basis for a fast, defensible decision.

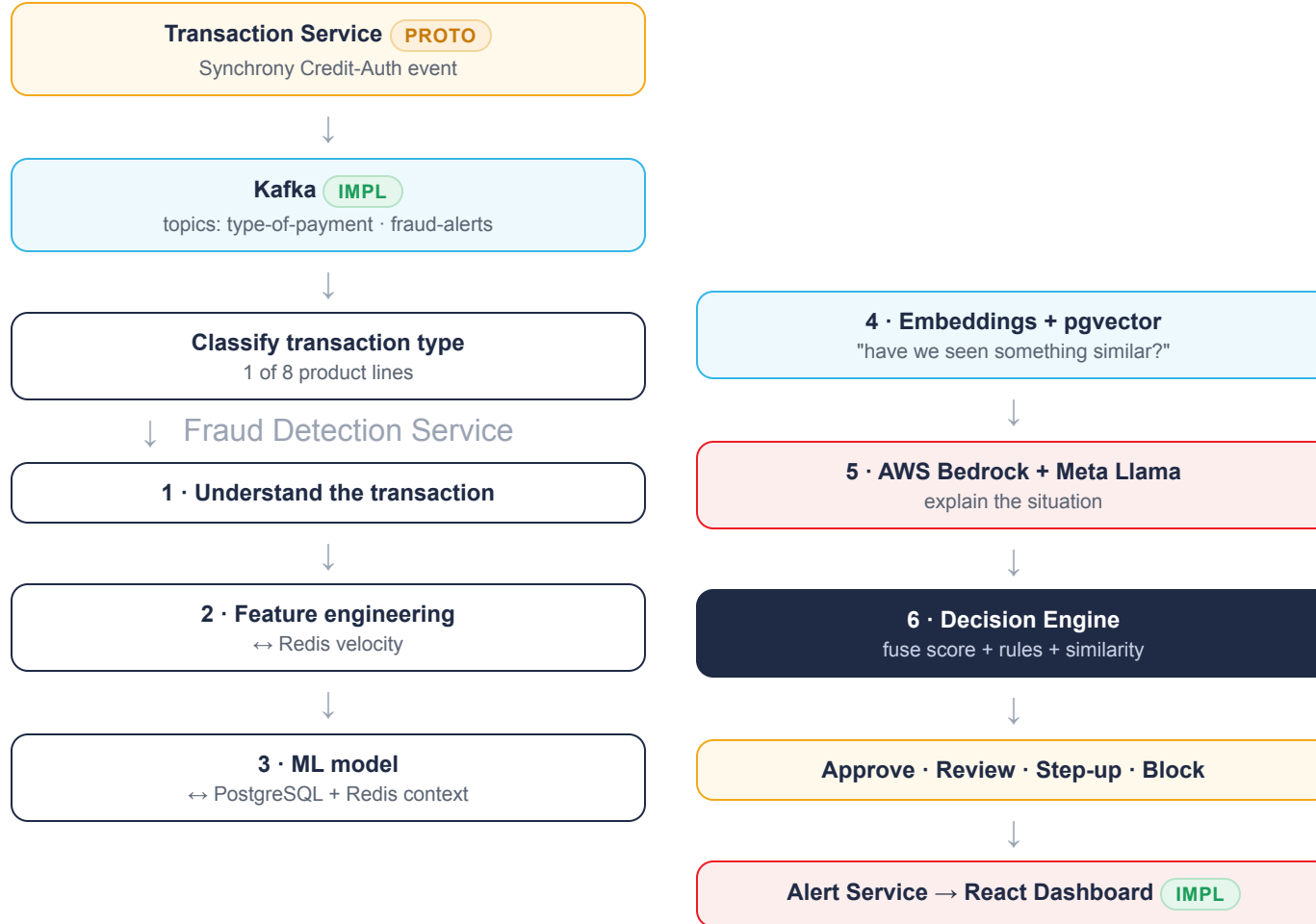
The core challenge

"How can we dynamically analyze every transaction, identify the appropriate fraud signals, detect known and unknown patterns, and make an explainable real-time decision?"

Our answer: make the pipeline **transaction-type-aware** and **evidence-first** — the system explains what it saw, not just what it concluded.



A layered, transaction-type-aware fraud pipeline



Why this order

Classify first, so only the fraud vectors that can occur for that product line are evaluated — the system never blindly applies one model to every transaction.

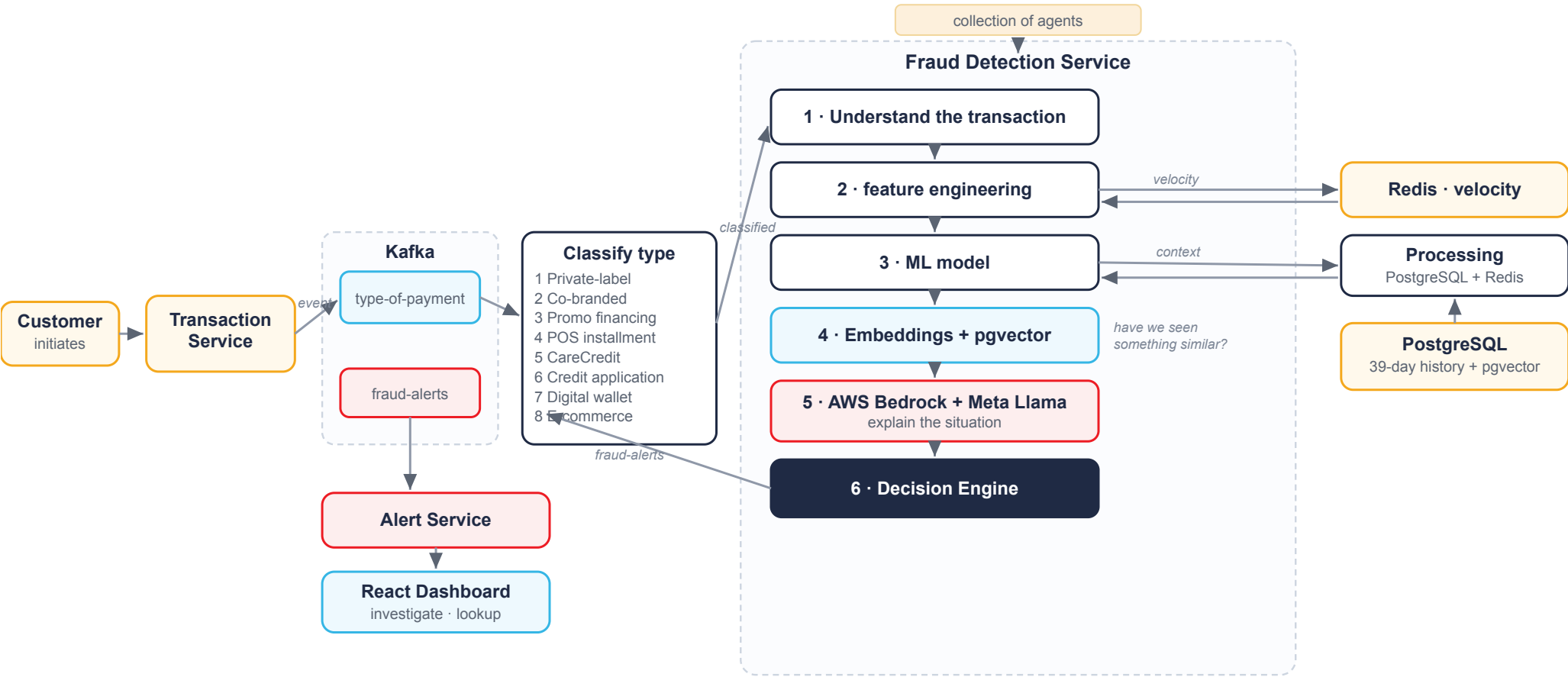
Context before scoring: PostgreSQL supplies the customer's 39-day history and Redis the hot velocity, so anomalies are judged relative to the customer.

Models decide, the LLM explains. Deterministic ML + rules + vector precedent own the risk; Bedrock only produces the reason. The Decision Engine makes the call.

A "collection of agents" (rule · ML · velocity · graph · vector) runs inside the detection service and feeds the Decision Engine.

ARCHITECTURE

End-to-end system architecture

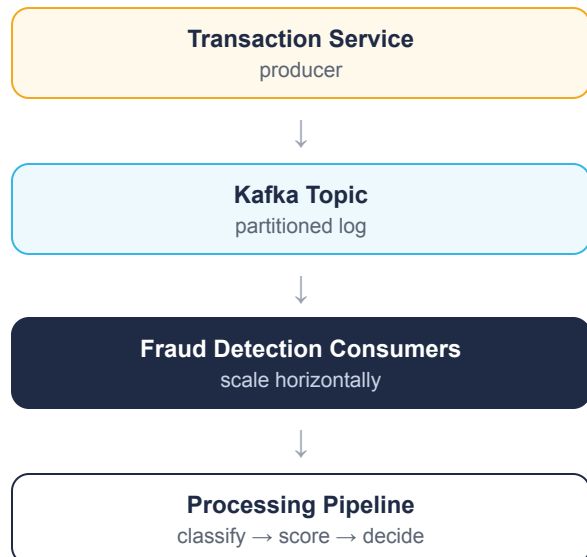


#	Step	Detail
1–2	Ingest	Transaction Service emits the Synchrony Credit-Auth event
3	Publish	Event → Kafka type-of-payment
4	Classify type	Identify the product line (1 of 8)
5	Understand + retrieve	Pull the account's 39-day history (PostgreSQL) + hot velocity (Redis)
6	Feature engineering	Build the 26-feature vector (amount, z-score, channel, CVV/AVS, MCC...)
7	ML model	RandomForest probability + IsolationForest anomaly
8	Embeddings + pgvector	Embed → kNN over 2,750 labelled cases → nearest precedents
9	Bedrock + Meta Llama	Structured evidence → human-readable reason
10	Decision Engine	Fuse score + rules + similarity → APPROVE / REVIEW / BLOCK
11–12	Act	Kafka fraud-alerts → Alert Service → React dashboard

```
{
  "account_id": "ACC-000468",
  "amount": 0.75,           // micro-charge
  "channel": "e_commerce",
  "entry_method": "cnp",
  "cvv_result": "no_match",
  "avs_result": "N",
  "status": "declined"
}
```

Type: Co-branded e-commerce · **Vector:** CB-2 card-testing
Signals: cvv_mismatch · avs_mismatch · small_amount_decline
Nearest cases: 5 / 8 labelled fraud (CB-2)
Decision: BLOCK / step-up · risk 1.00

Why Kafka? — decoupling and replay



What Kafka buys us

- **Decouples** transaction processing from fraud detection.
- High-throughput, **asynchronous** event streaming.
- Multiple independent **consumers** (scoring, audit, analytics).
- **Replay** — re-run history against a new model version.
- Producer is never coupled to, or blocked by, the fraud engine.

Topic organisation

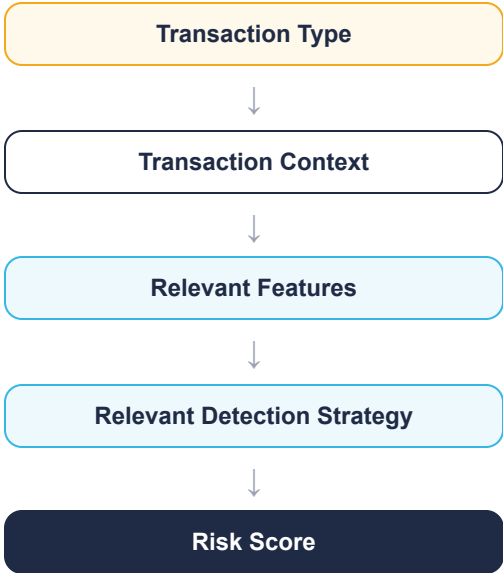
type-of-payment classified transaction events (producer → detection)

fraud-alerts high-risk decisions → Alert Service → dashboard

IMPL Kafka (KRaft) runs in the Docker stack. **PROTO** live consumer wiring is partial — detection currently runs synchronously via the API for the demo.

CORE IDEA

Dynamic, transaction-type-aware detection



Classification **activates only the fraud vectors that can occur** for that product line — not one model for everything.

Transaction type	Illustrative relevant signals
Private-label credit card	gift-card MCC, promo abuse, in-store entry mode
Co-branded / dual card	CVV/AVS mismatch, magstripe fallback, card-testing bursts
Promotional financing	promo-plan misuse, amount thresholds, velocity
POS installment loan	identity signals, first-payment default risk
CareCredit transaction	health-MCC pattern, provider anomalies
Real-time credit application	identity + application velocity, device/synthetic-ID history
Digital wallet payment	token provisioning age, device change, account-takeover
E-commerce checkout	CNP, shipping/billing mismatch, IP + device fingerprint

IMPL 8-type taxonomy + 11 observed fraud vectors; rule-map + ML select the active signals. **PROPOSED** per-type specialist sub-models.

FEATURES & BEHAVIORAL ANALYTICS

Signals grouped by what they capture

Customer

- Transaction frequency
- Average / z-scored amount
- Spending profile
- Account age
- Prior fraud alerts

Transaction

- Amount
- Timestamp / hour
- Payment & entry method
- Transaction type
- Merchant · MCC · channel

Behavioral

- Velocity
- Sudden spend change
- Unusual timing
- Location deviation
- Device change

Network

- Shared device
- Shared IP
- Shared merchant
- Repeated entities
- Transaction linkage

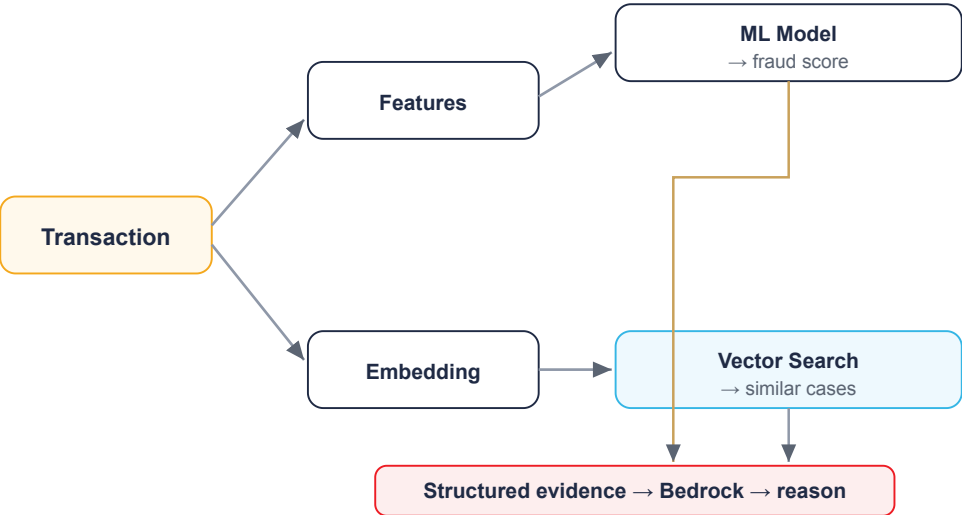
Behavioral baseline from 39-day history IMPL

Each account's prior 39 days live in PostgreSQL. On the 40th day we retrieve **only that account's** history (index-backed) to compute its baseline — mean spend, channel mix, first/last seen — so anomalies are judged relative to the customer, not a global average.

```
backend/app/ml_features.py
FEATURE_ORDER = [
    "amount","hour","amount_zscore",
    "cvv_no_match","avs_no_match",
    "acct_txn_count","acct_avg_amount",
    chan_*, entry_*, mcc_* ] # train==serve
```

AI COMPONENTS

ML + Embeddings + Vector search — distinct roles



ML model IMPL
RandomForest (fraud probability) + IsolationForest (anomaly) → the **core numerical risk**.

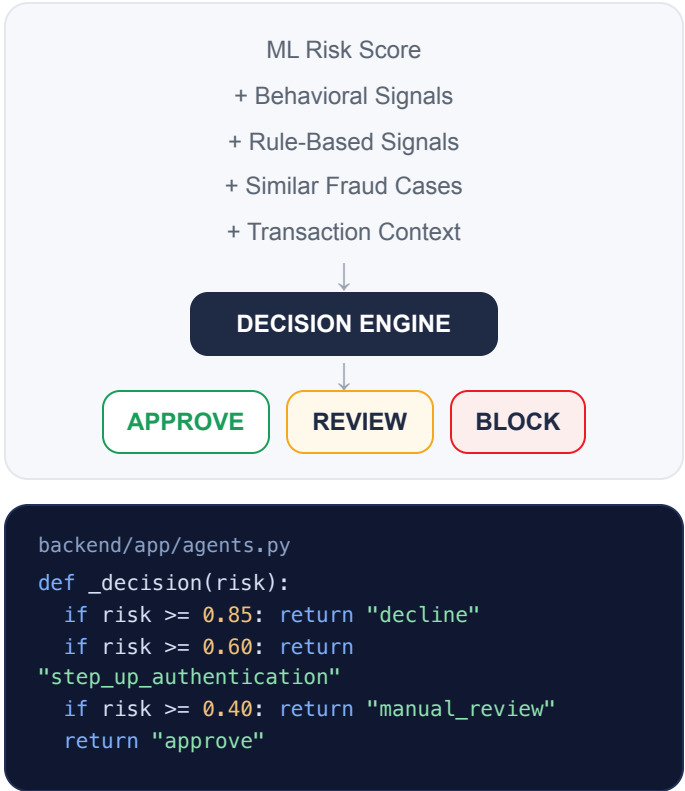
Embeddings IMPL
Scaled 26-dim behavioral vector — a transaction's behavioral fingerprint.

pgvector IMPL
kNN over **2,750** labelled cases (250 fraud + 2,500 legit) → nearest precedents + RAG.

LLM is not the detector
Deterministic models produce the risk. The LLM only does **reasoning, explanation and analyst assistance**.

DECISION ENGINE

Controlled fusion — the model decides, not the LLM



Guardrail: no uncontrolled financial decisions

The LLM never issues an approve/block — it returns text. The deterministic Decision Engine maps fused risk → action, keeping the money-path auditable and reproducible. A **consensus boost** raises risk when ≥2 agents agree.

Every decision is persisted for audit IMPL

- Risk score
 - Triggered signals
 - Rules / vectors
 - Similar cases
 - Decision + actor
 - Explanation
 - Timestamp
 - Model / version

Evidence, not just a "fraud" flag

Transaction

\$2,850

unseen-device purchase



RISK: HIGH

Final decision → **REVIEW / BLOCK**

Illustrative format; the live system emits the same evidence structure (see dashboard).

Why this was flagged — ranked evidence

1. Amount is **significantly above** the customer's historical average.
2. Originated from a **previously unseen device**.
3. **Multiple transactions** in a short time window (velocity).
4. **Location differs** from recent customer activity.
5. Similar historical transactions were **associated with fraud** (vector precedent).

Live evidence contract IMPL

/investigate and /lookup return: indicators, contributing signals, nearest labelled cases, ML score, actor + fraud vector, and a Bedrock reason — surfaced directly in the analyst UI.

Fraud investigation workspace IMPLEMENTED

synchrony

ANALYTICS

• Live · Postgres + pgvector + Redis

Overview

Fraudulent Accounts

Account Lookup

52,260

Transactions

85

Fraud accts

250

Alerts

188

Critical

CB-2

Top vector

Detected Fraudulent Accounts

ACC-000468	CB-2	24	100	\$598.59
ACC-000166	CB-2	22	100	\$23.25
ACC-000595	CB-2	17	100	\$15.00
ACC-000314	X-1	1	99	\$1,715

Fraud Account Lookup

Account ID

Txn ID

ACC-000468

FRAUDULENT · CB-2 · third-party
cvv mismatch · avs mismatch · small-amt decline

Live React + Tailwind dashboard (rendered layout), backed by real API calls over the PostgreSQL history store.

Top metrics

transactions analyzed · fraudulent accounts · fraud alerts · critical alerts · top vector (blocked / under-review counts & avg latency — PLANNED)

Fraud account → detail

Click any account: identifier · triggering transactions · who initiated (actor) · why flagged · indicators · ML score · 39-day context · recent history.

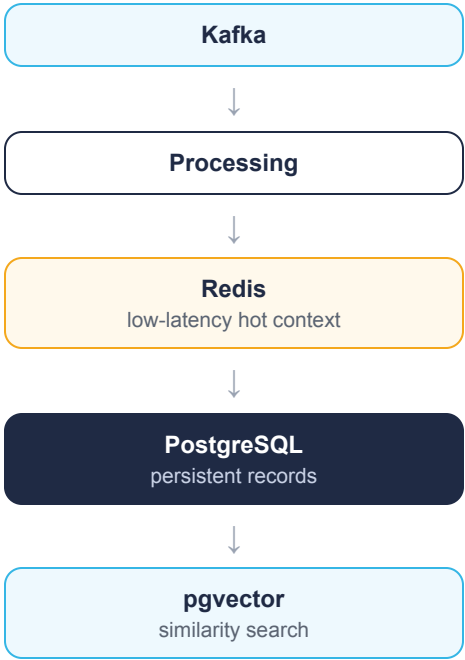
Fraud Lookup

Search by **primary key** — account_id / transaction_id — returns fraud status + evidence. Customer/merchant ID keys — PLANNED.

synchrony

BITS Pilani, Pilani Campus

Why Redis and PostgreSQL + pgvector



PostgreSQL

IMPL

- Customer + account data
- Transactions (39-day)
- Fraud decisions / alerts
- Model outputs
- Audit trail

1,000 accounts · 52,260 txns · indexed (account_id, event_time).

pgvector

IMPL

- Transaction embeddings
- Fraud embeddings
- kNN similarity

2,750 labelled case vectors (26-dim).

Redis

PROTO

- Hot customer context
- Recent-txn features
- Velocity counters
- Feature caching

Running in stack; velocity agent integration in progress.

Efficient context retrieval — never dump the whole DB to the model

On the 40th day we fetch **one account's** prior-39-day rows via the composite index, compute its baseline in-service, and pass only that context to features + embeddings. Fast and privacy-scoped.

Security

- ## Responsible AI

- ## Engineering

- ### Honesty note for evaluators

Constraints are stated, not hidden: the AWS Organization SCP blocks Secrets Manager / Cognito, so credentials are bcrypt-hashed in PostgreSQL for now; Anthropic Claude needs account verification, so the LLM layer runs on **Bedrock Meta Llama-3 8B**. Deployment choices, clearly labelled.

Layers in code — Frontend · Backend · Database

1 · Frontend REACT

my-react-app/src/components/SignInModal.jsx

```
const res = await fetch(
  `${API_BASE}/auth/login`, {
    method: "POST",
    headers: {"Content-Type":
      "application/json"},
    body: JSON.stringify(
      { email, password }),
  });
if (res.ok) {
  const d = await res.json();
  onSignIn(d.access_token, email);
}
```

React UI authenticates against the real API and stores the JWT for subsequent calls.

2 · Backend FASTAPI

backend/app/main.py

```
@app.post("/investigate")
def investigate(
    txn: TransactionIn,
    user: str = Depends(
        get_current_user)):
    return {"analyst": user,
        **_orchestrator.run(
            txn.model_dump())}

@app.get("/accounts/fraud")
def accounts_fraud(user=Depends(
    get_current_user)):
    return {"accounts":
        analytics.fraud_accounts()}
```

API-first: typed, JWT-protected endpoints; the orchestrator runs the multi-agent pipeline.

3 · Database PG + PGVECTOR

alembic/.../0001_initial.py · models.py

```
op.execute(
    "CREATE EXTENSION IF NOT "
    "EXISTS vector")

# models.py
embedding = Column(
    Vector(EMBEDDING_DIM))

# analytics.py — KNN
"ORDER BY embedding <=> "
"[...]"::vector LIMIT :k"
```

pgvector enabled by migration; embeddings stored as a vector column; similarity via the <=> operator.

Layers in code — AI · Cloud · Security

4 · AI Layer BEDROCK

backend/app/explain.py

```
rt = boto3.client(
    "bedrock-runtime",
    region_name=AWS_REGION)

# guardrail: strip PII first
def redact(txn):
    safe["ip_address"]="***"
    safe["masked_pan"]=pan[-4:]
    return safe

rt.invoke_model(
    modelId=model_id, body=...)
```

Bedrock (Meta Llama) for explanations; embeddings + pgvector for similarity; PII redacted before any LLM call; template fallback.

5 · Cloud Layer AWS

docker-compose.yml · config.py

```
# creds from host IAM role,
# never in the image
volumes:
  - ${HOME}/.aws:/root/.aws:ro
environment:
  AWS_REGION: ${AWS_REGION}
  BEDROCK_MODEL_ID: ${...}

# health/monitoring hooks
@app.get("/health/deps") # pg/redis/kafka
```

Same Docker stack targets a single EC2; Bedrock live; health endpoints for monitoring. PROPOSED: RDS, Secrets Manager, CloudWatch (see docs/aws-postgres.md).

6 · Security AUTH

backend/app/security.py · schemas.py

```
# bcrypt verify vs Postgres
bcrypt.checkpw(pw.encode(),
    user.password_hash.encode())
jwt.encode({"sub":sub,
    "exp":now+TTL}, JWT_SECRET)

# input validation
class LoginIn(BaseModel):
    email: str = Field(
        min_length=3,max_length=254)
```

No hardcoded secret (JWT key from env, credential in DB); bcrypt + JWT; Pydantic validates every request.

Prototype results and the path to production

52,260

transactions loaded & scorable

~15 ms

ML + rule scoring / txn (measured)

~3.1 s

full /investigate incl. Bedrock LLM (measured)

2,750

pgvector labelled cases

Prototype evaluation

IN-SAMPLE

Metric (risk ≥ 0.50)	Value	Metric	Value
Recall (245 / 250)	0.98	Precision	1.00
F1	0.99	False-positive rate	0.000

Read critically: models trained on this **synthetic** dataset with no held-out split, and injected fraud carries clearly-encoded signals — so these are **in-sample** figures confirming the pipeline learns the intended patterns, **not** generalization estimates. Stratified split + adversarial evaluation are the next steps.

How it scales

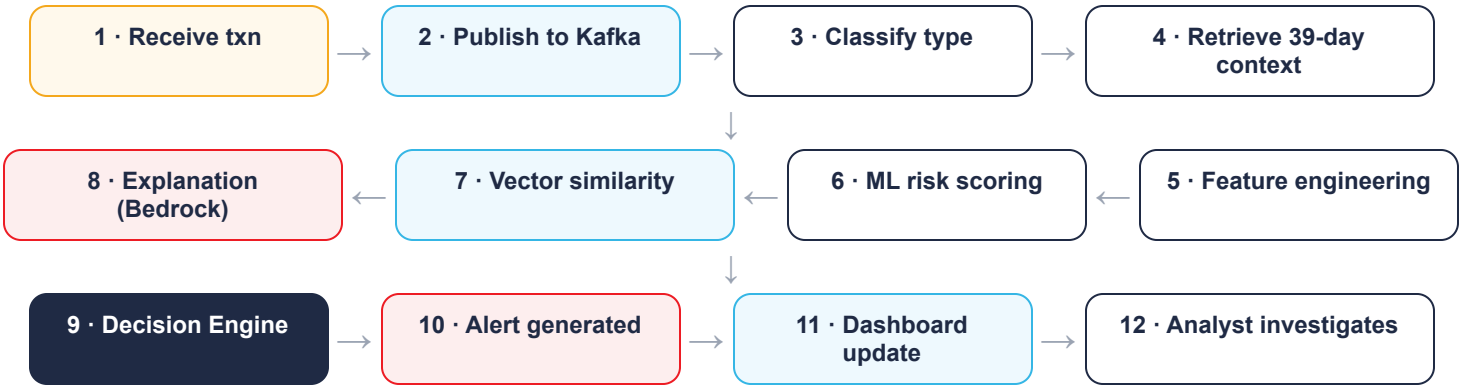
- Kafka horizontal scaling · multiple consumers
- Stateless FastAPI services
- Redis caching · PostgreSQL indexing
- pgvector ANN similarity
- AWS deployment · model versioning · observability

Future roadmap

- Online / incremental learning
- Graph-based fraud-ring detection
- Advanced behavioral profiling
- Agent-assisted investigation
- Continuous monitoring + feedback loop
- Cross-merchant fraud intelligence

DEMO FLOW

From event to explainable decision



"From transaction event → contextual intelligence → explainable risk → real-time decision."